

Math Nuggets for GEN AI

Parameter-Efficient Fine-Tuning: Mathematical Foundation of QLoRA for LLMs

By Dr. Anish Roychowdhury, Plaksha University

September 25, 2025

Beyond Full Fine-Tuning: The Efficiency Challenge

Traditional fine-tuning of large language models (LLMs) updates all parameters, leading to several fundamental limitations:

Consider the task: "Fine-tune a 7B parameter LLM for domain-specific question answering."

Full fine-tuning faces the following critical challenges:

- **Resource Intensity:** Requires GPU memory proportional to model size
- **Storage Overhead:** Multiple fine-tuned copies for different tasks
- **Training Instability:** Catastrophic forgetting of pre-trained knowledge
- **Deployment Cost:** High inference latency with modified weights

Why PEFT?

Parameter-Efficient Fine-Tuning (PEFT) solves these limitations by updating only a small subset of parameters while keeping the base model frozen, combining the adaptability of fine-tuning with the efficiency of few-shot learning.

PEFT Architecture: Mathematical Framework

PEFT methods, particularly QLoRA, enable efficient adaptation by introducing low-rank adapters and quantization.

Core Mathematical Formulation:

Let $\mathbf{W} \in \mathbb{R}^{d \times k}$ be a pre-trained weight matrix in the LLM, and $\Delta\mathbf{W}$ be the update during fine-tuning. In LoRA (foundation of QLoRA), we decompose:

$$\Delta\mathbf{W} = \mathbf{B}\mathbf{A} \tag{1}$$

Where:

- $\mathbf{A} \in \mathbb{R}^{r \times k}$ is a low-rank matrix (initialized randomly)
- $\mathbf{B} \in \mathbb{R}^{d \times r}$ is a low-rank matrix (initialized to zero)
- $r \ll \min(d, k)$ is the rank hyperparameter

The adapted forward pass becomes:

$$\mathbf{h} = (\mathbf{W} + \mathbf{B}\mathbf{A})\mathbf{x} = \mathbf{W}\mathbf{x} + \mathbf{B}(\mathbf{A}\mathbf{x}) \tag{2}$$

QLoRA Extension: QLoRA adds 4-bit quantization to the base model weights:

$$\mathbf{W}_q = \text{Quantize}(\mathbf{W}, 4\text{-bit}) \tag{3}$$

$$\mathbf{h} = \text{Dequantize}(\mathbf{W}_q)\mathbf{x} + \mathbf{B}(\mathbf{A}\mathbf{x}) \tag{4}$$

This reduces memory footprint while maintaining performance through double de-quantization and paged optimizers.

Two-Stage Process: PEFT decomposes the adaptation problem into:

- **Offline Preparation** - quantize base model and initialize adapters,
- **Online Fine-Tuning** - update only adapter parameters on task data.

This decomposition allows efficient training on consumer hardware while preserving base model integrity.

Low-Rank Adaptation: Matrix Decomposition Mathematics

QLoRA builds on LoRA's use of low-rank matrix approximations for parameter efficiency.

Adapter Initialization:

$$\mathbf{A} \sim \mathcal{N}(0, \sigma^2) \quad (\text{Gaussian initialization}) \quad (5)$$

$$\mathbf{B} = \mathbf{0} \quad (\text{Zero initialization for identity start}) \quad (6)$$

Parameter Count Reduction: Original parameters: $d \times k$

LoRA parameters: $r \times k + d \times r = r(d + k)$

Efficiency ratio: $\frac{r(d+k)}{dk} \approx \frac{2r}{\min(d,k)}$ (typically $< 1\%$ for $r = 8, d = k = 4096$)

Quantization Mathematics: 4-bit NormalFloat (NF4) quantization maps weights to $[-1,1]$ interval:

$$\mathbf{W}_q[i, j] = \text{round} \left(\frac{\mathbf{W}[i, j] - \mu}{\sigma} \times 7.5 \right) + 8 \quad (7)$$

Where values are stored as 4-bit integers (0-15), with dequantization:

$$\text{Dequant}(\mathbf{W}_q[i, j]) = q_{\text{levels}}[\mathbf{W}_q[i, j]] \times c_1 + c_2 \quad (8)$$

Using pre-computed quantization levels for optimal information preservation.

Figure 1 shows the LoRA process in a schematic way.

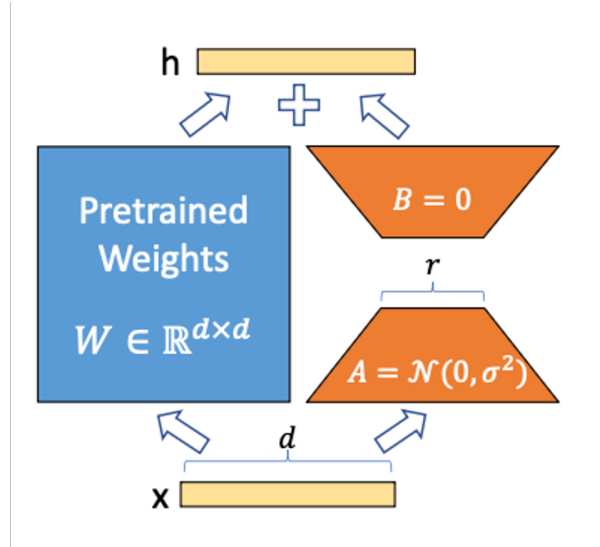


Figure 1: Schematic of LoRA Process [1]

QLoRA Pipeline Architecture

The complete QLoRA system architecture is illustrated in Figure 1, showing the flow from base model to fine-tuned adapter generation.

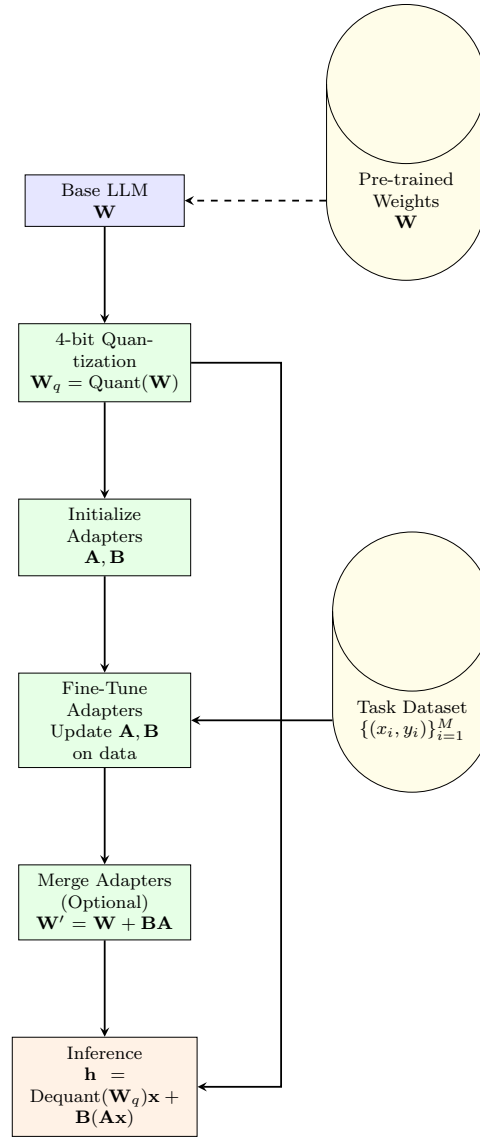


Figure 2: QLoRA pipeline showing quantization, adapter initialization, and fine-tuning

Worked Example: QLoRA for Linear Layer Adaptation

Let's implement QLoRA for a simple linear layer with input "Adapt a layer for text classification" and a small weight matrix.

Setup:

- Original dimension: $d = k = 4$
- Rank: $r = 2$
- Quantization: 4-bit (simplified to rounding for demo)

Original Weight Matrix:

$$\mathbf{W} = \begin{bmatrix} 1.2 & 0.5 & -0.3 & 0.8 \\ 0.4 & 1.1 & 0.6 & -0.2 \\ -0.7 & 0.3 & 1.4 & 0.5 \\ 0.9 & -0.4 & 0.2 & 1.3 \end{bmatrix} \quad (9)$$

Step 1: 4-bit Quantization (Simplified)

Map to $[-1,1]$ and quantize to 4 levels: -1, -0.33, 0.33, 1

$$\mathbf{W}_q \approx \begin{bmatrix} 1 & 0.33 & -0.33 & 1 \\ 0.33 & 1 & 0.33 & -0.33 \\ -1 & 0.33 & 1 & 0.33 \\ 1 & -0.33 & 0.33 & 1 \end{bmatrix} \quad (10)$$

(Actual NF4 uses more precise levels; this is illustrative.)

Step 2: Initialize Adapters

$$\mathbf{A} = \begin{bmatrix} 0.1 & -0.05 \\ 0.2 & 0.1 \\ -0.1 & 0.15 \\ 0.05 & -0.2 \end{bmatrix}^T, \quad \mathbf{B} = \begin{bmatrix} 0 & 0 \\ 0 & 0 \\ 0 & 0 \\ 0 & 0 \end{bmatrix} \quad (11)$$

(Note: $\mathbf{A}$ is $r \times k = 2 \times 4$, transposed for display.)

Step 3: Fine-Tune Adapters

Assume a small update after training:

$$\mathbf{A}' = \begin{bmatrix} 0.12 & -0.03 \\ 0.18 & 0.12 \\ -0.08 & 0.17 \\ 0.07 & -0.18 \end{bmatrix}^T, \quad \mathbf{B}' = \begin{bmatrix} 0.05 & 0.02 \\ -0.03 & 0.04 \\ 0.01 & -0.02 \\ 0.03 & 0.01 \end{bmatrix} \quad (12)$$

Step 4: Compute Adapted Weights (If Merged)

$$\Delta \mathbf{W} = \mathbf{B}' \mathbf{A}' = \begin{bmatrix} 0.05 & 0.02 \\ -0.03 & 0.04 \\ 0.01 & -0.02 \\ 0.03 & 0.01 \end{bmatrix} \begin{bmatrix} 0.12 & 0.18 & -0.08 & 0.07 \\ -0.03 & 0.12 & 0.17 & -0.18 \end{bmatrix} \quad (13)$$

$$\approx \begin{bmatrix} 0.0054 & 0.0114 & -0.0006 & -0.0001 \\ -0.0048 & -0.0006 & 0.0093 & -0.0028 \\ 0.0018 & 0.0022 & -0.0042 & 0.0053 \\ 0.0033 & 0.0057 & -0.0019 & 0.0019 \end{bmatrix} \quad (14)$$

Step 5: Forward Pass with Input $\mathbf{x}$

Let $\mathbf{x} = \begin{bmatrix} 1 \\ 0 \\ 1 \\ 0 \end{bmatrix}$

Original: $\mathbf{W}\mathbf{x} \approx \begin{bmatrix} 0.9 \\ 1.0 \\ 0.7 \\ 1.1 \end{bmatrix}$

Adapted: $\text{Dequant}(\mathbf{W}_q)\mathbf{x} + \mathbf{B}'(\mathbf{A}'\mathbf{x}) \approx \begin{bmatrix} 0.67 \\ 0.0 \\ 0.0 \\ 0.67 \end{bmatrix} + \begin{bmatrix} 0.0048 \\ -0.0054 \\ 0.0020 \\ 0.0030 \end{bmatrix}$

Step 6: Efficiency Calculation

Original parameters: $4 \times 4 = 16$

QLoRA parameters: $2 \times 4 + 4 \times 2 = 12$ (75% of original, but scales better for large models).

Mathematical Insight: The low rank $r = 2$ captures task-specific adaptations in a subspace, reducing parameters while the quantization saves memory. For large LLMs, this enables fine-tuning on single GPUs.

Summary

QLoRA revolutionizes LLM adaptation by mathematically combining quantization with low-rank adapters:

- Enables fine-tuning of billion-parameter models on limited hardware
- Prevents catastrophic forgetting through frozen base weights
- Allows easy merging and deployment of adapters
- Maintains near full fine-tuning performance with $< 1\%$ trainable parameters

The mathematical elegance lies in the low-rank decomposition that exploits the intrinsic low-dimensional structure of fine-tuning updates. This approach addresses fundamental limitations of full fine-tuning while preserving model capabilities, establishing QLoRA as a cornerstone technique for efficient LLM customization.

Practical Implementation Notes: Real-world QLoRA systems must address several mathematical challenges:

- **Quantization Error** - mitigating precision loss through double dequantization
- **Rank Selection** - tuning r for optimal efficiency-performance tradeoff
- **Multi-Adapter Fusion** - combining multiple LoRA adapters for multi-task learning

References

- [1] DBLP:journals/corr/abs-2106-09685